

How to Help Transformers Count

Elaine Wu (ew457@cornell.edu)

Jillian Chong (jc2886@cornell.edu)

Lina Hao (lh657@cornell.edu)

Code contained at the GitHub repository here: <https://github.com/JillianChong/cs4782-final>

INTRODUCTION

Arithmetic is a crucial component of human reasoning and serves as a useful benchmark for evaluating the reasoning abilities of transformer-based language models. Because these models process text rather than explicit mathematical structure, it is unclear whether they truly learn arithmetic rules or simply memorize patterns from training data.

This project reproduces the findings from “Investigating the Limitations of Transformers with Simple Arithmetic” by Rodrigo Nogueira, Zhiying Jiang, and Jimmy Lin. The paper investigates how different number representations affect transformer performance on addition and subtraction tasks. The authors train the T5 (Text-to-Text Transfer Transformer) models on synthetic arithmetic datasets using different tokenization schemes, including subword representation (e.g. “456”), character-level representation (e.g. “4 5 6”), and position-token representation (e.g. “4 10e2 5 10e1 6 10e0”).

Their experiments show that explicit positional tokens drastically improved arithmetic accuracy in comparison to subword and character-level representations, especially for longer numbers.

Chosen Result

We specifically aimed to reproduce the finding that using position-token representations for addition and subtraction operations allowed the model to perform at a higher accuracy than subword and character-level representations. This result was the paper’s main contribution and demonstrates that model performance depends on numerical representation, rather than model scale or training size alone.

We chose this result because it directly supports the paper’s main argument: that transformer limitations on arithmetic stem less from the model architecture itself and more from how positional information is represented in numerical inputs.

Figure 1 from the original paper illustrates the performance of each representation on addition across different digit lengths. While many representations begin with near-perfect accuracy, non-position token representations’ accuracies decline far more rapidly than position-token representations as digit length increases.

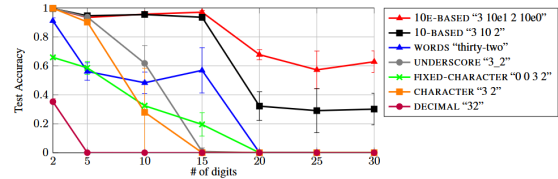


Figure 1 (Nogueira et al., 2021): Accuracy of different number representations on the addition task.

In addition to reproducing this core result, we also investigated length generalization by evaluating how models perform on addition problems using numbers of unseen lengths. This experiment aimed to test how transformers generalize arithmetic rules beyond their training distribution.

Methodology

We reproduce the experimental setup from the paper by fine-tuning a pre-trained Transformer model on addition and subtraction. Specifically, we use a pre-trained T5 model, following the original paper’s approach of fine-tuning an existing T5 model rather than training from scratch. In our experiments, we use both T5-small and T5-base due to computational constraints, whereas the paper also explores larger T5 variants.

The dataset is generated synthetically, where each example contains two integers paired with their arithmetic result (addition or subtraction). Training and development datasets are created using balanced sampling. For a chosen maximum digit length D , examples are created by first sampling a digit length $d \in [2, D]$, then uniformly sampling both operands from the range corresponding to d -digit numbers. This produces a roughly even distribution of examples across different number lengths from 2 to D .

Testing datasets are generated using random sampling, where operands are sampled uniformly from the full range $[0, 10^D - 1]$. This produces a test distribution that is dominated by larger D -digit numbers, allowing evaluation on the most difficult examples seen during training.

To study the effect of numerical representation, we experiment with several encoding schemes, including digit-level, character-level, and positional encodings. For a given experiment, all operands and corresponding arithmetic results are formatted according to the selected numerical representation.

These results vary how explicitly numerical structure is presented to the model.

We train separate models for increasing maximum digit lengths, specifically $D \in \{2, 5, 10, 15, 20, 25, 30\}$. Each model is trained only on numbers up to its assigned digit length.

Models are trained using the HuggingFace Seq2SeqTrainer with cross-entropy loss, masking padding tokens during training. During evaluation, we use greedy decoding and measure performance using exact-match accuracy on our test sets, following the evaluation protocol of the original paper.

For the additional experiment on length generalization, we used the T5-base models trained on $D \in \{2, 5, 10\}$ and evaluated their accuracy on 11 synthetically generated testing sets, each containing inputs of a fixed digit length $k \in [2, 12]$. For example, one test set contains only 2-digit numbers, another only 3-digit inputs, and so on.

Results & Analysis

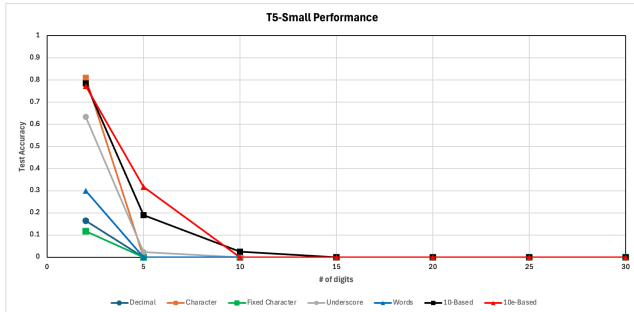


Figure 2: Performance of Representations on T5-Small.

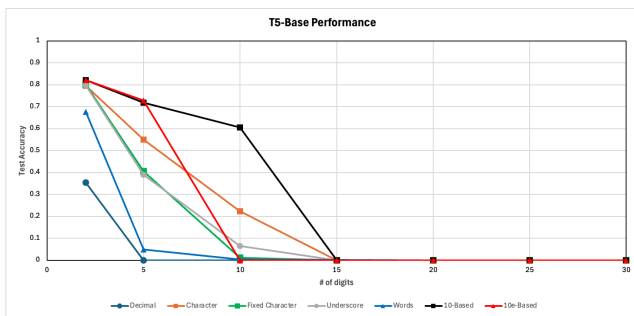


Figure 3: Performance of Representations on T5-Base.

The original paper demonstrated that position token representations maintained near-perfect accuracy up to 15 digits on addition tasks, before gradually declining at longer digit lengths, still substantially outperforming all other representations at every digit length. Our re-implementation, constrained to T5-small and T5-base due to our challenges with Google Colab hardware memory limits, showed drastic performance decreases beyond 15 digits. We

attribute this gap to limitations on batch sizes and input and output sequence length limits forced by memory constraints. For longer numbers, models need many more tokens to be able to represent the full number in its necessary representation.

While our accuracy values were lower than those reported in the original paper, our results in Figures 1 and 2 replicate the same qualitative trend across both the T5-small and T5-base models: position token representations (10e-based and 10-based) outperformed representations without positional tokens, with performance degrading more steeply for non-position token representations as the number of digits increased. These results support the paper’s argument that the surface form of number representations significantly impacts a transformer’s ability to learn arithmetic, and that tokenization is a component of current transformer designs that may need improvement.

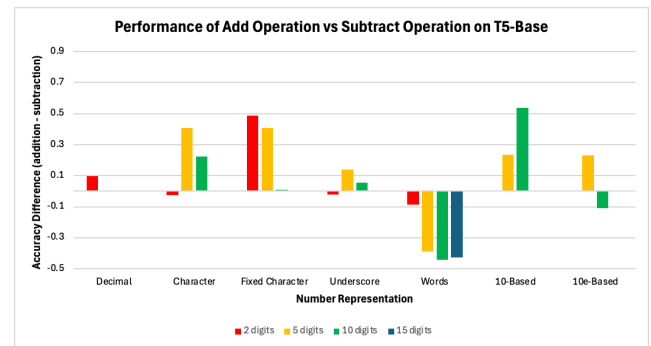


Figure 4: Addition Task v.s. Subtraction Task Accuracy.

Figure 4 shows the difference in accuracy between addition and subtraction tasks across representations on T5-base. The more positive the difference, the better the model performed on the addition task compared to the subtraction task. We found that all representations, except word-based, performed better on addition than subtraction. Our explanation for this is that subtraction is inherently harder due to borrowing and non-commutativity, but word-based representations may alleviate these challenges by introducing clearer semantic and positional cues, making operand order and negatives more explicit. The linguistic structure may help the model capture these patterns more effectively than other, more compact encodings.

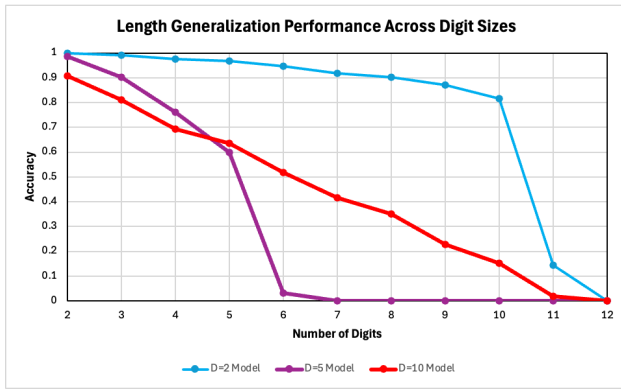


Figure 5: T5-Base Generalization Performance.

Across all models, Figure 5 shows accuracy decreases as input digit length increases, indicating limited length generalization in addition tasks. The $D=2$ model generalizes relatively well to longer inputs likely because its constrained training distribution encourages the model to learn digit-wise arithmetic patterns rather than length-specific patterns. The $D=5$ and $D=10$ models show sharp performance collapse beyond their training range, likely due to the models relying more heavily on length-specific heuristics rather than learning length-invariant arithmetic rules.

These results reinforce the paper’s claims that transformer limitations stem mainly from representation and how they learn, rather than from model capacity alone.

Reflections

Our re-implementation confirms that tokenization significantly impacts transformer performance, as large performance differences arose from representation choices rather than changes to model architecture. Our results also demonstrated that generalization remains a substantial challenge. Performance drops significantly once inputs exceed the training range, suggesting models tend to focus more on numerical length rather than learning arithmetic rules. Although our experiments were constrained by model input and output sequence length limits, the overall trends remained consistent with the original paper, suggesting that the core findings are robust even under practical computational constraints.

The paper and our findings highlight that improving performance depends not just on using larger models, but on structuring inputs well. The way information is represented can determine whether a model learns real patterns or surface-level cues. Good encoding is just as important as model scale.

Future work could explore hybrid numerical representations that combine digit-level, character-level, and positional encodings to better capture fine-grained digit-level relationships and overall number scale. Another promising direction is extending these ideas to multi-step arithmetic tasks, potentially supported by chain-of-thought-style reasoning to help models explicitly track intermediate computation steps. Finally, we could also explore other explicit positional representations that help models handle carry operations and digit order, like aligning numbers by place value and encoding them as columns. This investigation could lead to more robust generalization across longer inputs.

References

1. Nogueira, R., Jiang, Z., & Lin, J. (2021). Investigating the limitations of transformers with simple arithmetic tasks. arXiv. <https://arxiv.org/abs/2102.13019>
2. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2019). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. arXiv:1910.10683. <https://arxiv.org/abs/1910.10683>
3. Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew. (2019). HuggingFace’s Transformers: State-of-the-art Natural Language Processing. CoRR abs/1910.03771 (2019). arXiv:1910.03771 <http://arxiv.org/abs/1910.03771>